DISEÑO DE ALGORITMOS

ASESOR:
ING. GERARDO ACEVEDO VEGA

UNIDAD 3 · ACTIVIDAD COMPLEMENTARIA 2

IDENTIFICADORES, PALABRAS RESERVADAS Y ESTRUCTURA DE UN PROGRAMA EN C

21 DE OCTUBRE DE 2026

GRUPO 8191 · INFORMÁTICA

Alumno: Mauro Noel Sánchez Campos

Universidad Nacional Autónoma de México

Facultad de Contaduría y Administración

Introducción

Antes de escribir cualquier programa en C hay que saber dos cosas básicas: cómo nombrar correctamente las variables y funciones (identificadores) y cómo se organiza un programa. Si un identificador no cumple las reglas, el compilador marca un error antes de que el programa llegue a ejecutarse. En esta actividad clasifico una lista de identificadores, explico qué son las palabras reservadas, señalo las partes de un programa sencillo y propongo buenas prácticas para nombrar identificadores (Kanagusico et al., 2026).

1. Clasificación de identificadores

En C, un identificador debe cumplir estas reglas: comenzar con una letra o con guion bajo (_); después puede tener letras, dígitos y guiones bajos; no puede tener espacios ni símbolos como `-`, `$` o `%`; no puede ser una palabra reservada; y el lenguaje distingue mayúsculas de minúsculas, así que `edad` y `Edad` son identificadores distintos (Kernighan y Ritchie, 1991).

Identificador	Clasificación	Explicación
`edad`	**VÁLIDO**	Solo tiene letras y empieza con letra.
`2numero`	**NO VÁLIDO**	Empieza con un dígito; el compilador lo leería como un número.
`_total`	**VÁLIDO**	Puede empezar con guion bajo. (Por convención, los nombres que empiezan con _ se suelen reservar para la biblioteca estándar, así que conviene evitarlos.)
`nombre completo`	**NO VÁLIDO**	Tiene un espacio; el compilador lo vería como dos identificadores separados. Se podría escribir `nombreCompleto` o `nombre_completo`.
`precio$`	**NO VÁLIDO**	El símbolo $ no forma parte de los caracteres permitidos por el estándar de C. Algunos compiladores, como GCC, lo aceptan como extensión, pero el código deja de ser portable.
`int`	**NO VÁLIDO**	Es una palabra reservada (tipo de dato entero).
`promedioFinal`	**VÁLIDO**	Solo letras; usa estilo *camelCase*, que además lo hace legible.
`num-alumnos`	**NO VÁLIDO**	El guion medio es el operador de resta; el compilador lo interpreta como `num - alumnos`.
`Año1`	**NO VÁLIDO**	La letra ñ no pertenece al conjunto básico de caracteres de C (letras del alfabeto inglés). Compiladores recientes aceptan caracteres Unicode, pero no es portable; lo correcto es `Anio1`.
`while`	**NO VÁLIDO**	Es una palabra reservada (estructura de repetición).

Para comprobar la tabla compilé cada caso con GCC. `2numero`, `nombre completo`, `int`, `num-alumnos` y `while` generaron error. `precio$` y `Año1` compilaron en GCC porque este compilador los admite como extensión, por eso los marco como no válidos según el estándar, que es lo que garantiza que el programa funcione en cualquier compilador.

2. Palabras reservadas

Una **palabra reservada** (o palabra clave) es una palabra que el lenguaje ya tiene definida con un significado especial, como un tipo de dato o una estructura de control. No puede usarse como identificador porque el compilador no podría distinguir si nos referimos a la instrucción del lenguaje o a nuestra variable: si escribiéramos `int while = 5;`, el compilador esperaría el inicio de un ciclo y marcaría error de sintaxis. Algunas palabras reservadas de C son:

`auto`	`extern`	`sizeof`
`break`	`float`	`static`
`case`	`for`	`struct`
`char`	`goto`	`switch`
`const`	`if`	`typedef`
`continue`	`int`	`union`
`default`	`long`	`unsigned`
`do`	`register`	`void`
`double`	`return`	`volatile`
`else`	`short`	`while`
`enum`	`signed`	

Son las 32 palabras reservadas del estándar original de C (ANSI C89); las versiones posteriores agregaron otras como `inline`, `restrict` y `_Bool`.

3. Partes de un programa sencillo en C

Uso como ejemplo un programa que calcula el promedio de tres calificaciones:

```c
#include <stdio.h>   // (1)
#define MATERIAS 3

int main(void)   // (2)
{
    float cal1, cal2, cal3, promedio;  // (3)

    printf("Escribe tres calificaciones: ");  // (4)
    scanf("%f %f %f", &cal1, &cal2, &cal3);
    promedio = (cal1 + cal2 + cal3) / MATERIAS;
    printf("Tu promedio es: %.2f\n", promedio);

    return 0;   // (5)
}
```

Parte	Qué hace en el programa
1. Directivas del preprocesador	Empiezan con # y se procesan antes de compilar. `#include <stdio.h>` incluye la biblioteca de entrada y salida, que contiene `printf` y `scanf`; `#define` crea la constante `MATERIAS`.
2. Función principal	`int main(void)` es el punto donde empieza la ejecución; todo programa en C debe tener una. Las llaves `{ }` delimitan su bloque.
3. Declaraciones	Reservan memoria para las variables e indican su tipo: aquí, cuatro variables de tipo `float`.
4. Cuerpo	Instrucciones que resuelven el problema: pedir datos, leerlos, calcular el promedio y mostrarlo.
5. Valor de retorno	`return 0;` termina `main` y le indica al sistema operativo que el programa terminó correctamente.

4. Buenas prácticas para nombrar identificadores

1. **Usar nombres descriptivos.** `promedioFinal` dice qué guarda; `pf` o `x` obligan a adivinar. Un buen nombre hace que el código se explique solo.
2. **Seguir un estilo consistente.** Elegir uno, ya sea *camelCase* (`numAlumnos`) o *snake_case* (`num_alumnos`), y usarlo en todo el programa. Para constantes se acostumbra usar mayúsculas (`MATERIAS`, `IVA`).
3. **Evitar caracteres problemáticos y nombres confusos.** No usar acentos, `ñ` ni símbolos; no empezar con guion bajo; y no usar nombres que solo difieren en mayúsculas (`total` y `Total`), porque es fácil confundirlos.

Conclusión

Las reglas de los identificadores parecen simples, pero varios casos de la lista engañan: `num-alumnos` parece un nombre, pero el compilador lo lee como una resta, y `precio$` o `Año1` compilan en GCC aunque no son correctos según el estándar. Esto me dejó claro que "que compile" no siempre significa "que esté bien". También me sirvió identificar las partes de un programa, porque es la estructura que voy a repetir en todos los programas del curso.

Referencias

Cairó, O. (2005). *Metodología de la programación: algoritmos, diagramas de flujo y programas* (3a ed.). Alfaomega.

Kanagusico Hernández, E. D., Moreno García, R. C., y Hernández Sánchez, E. (2026). *Diseño de algoritmos* [Apunte digital, Licenciatura en Informática]. Universidad Nacional Autónoma de México, Facultad de Contaduría y Administración.

Kernighan, B. W., y Ritchie, D. M. (1991). *El lenguaje de programación C* (2a ed.). Prentice Hall Hispanoamericana.